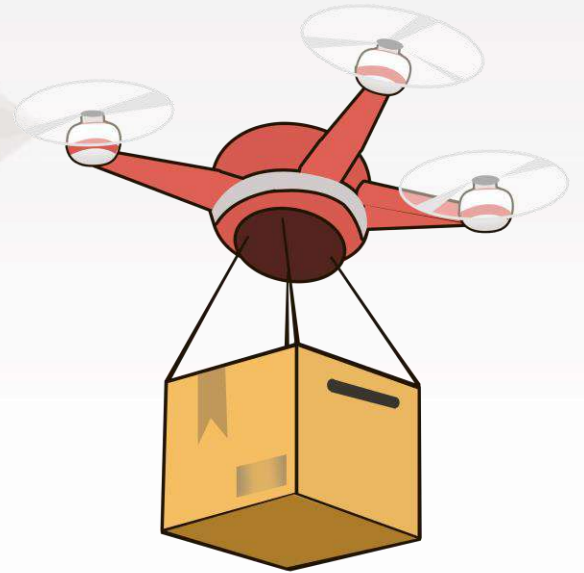




DRONE LIVREUR DE COLIS

RECHERCHE D'UN CHEMIN
OPTIMAL ÉNERGÉTIQUEMENT

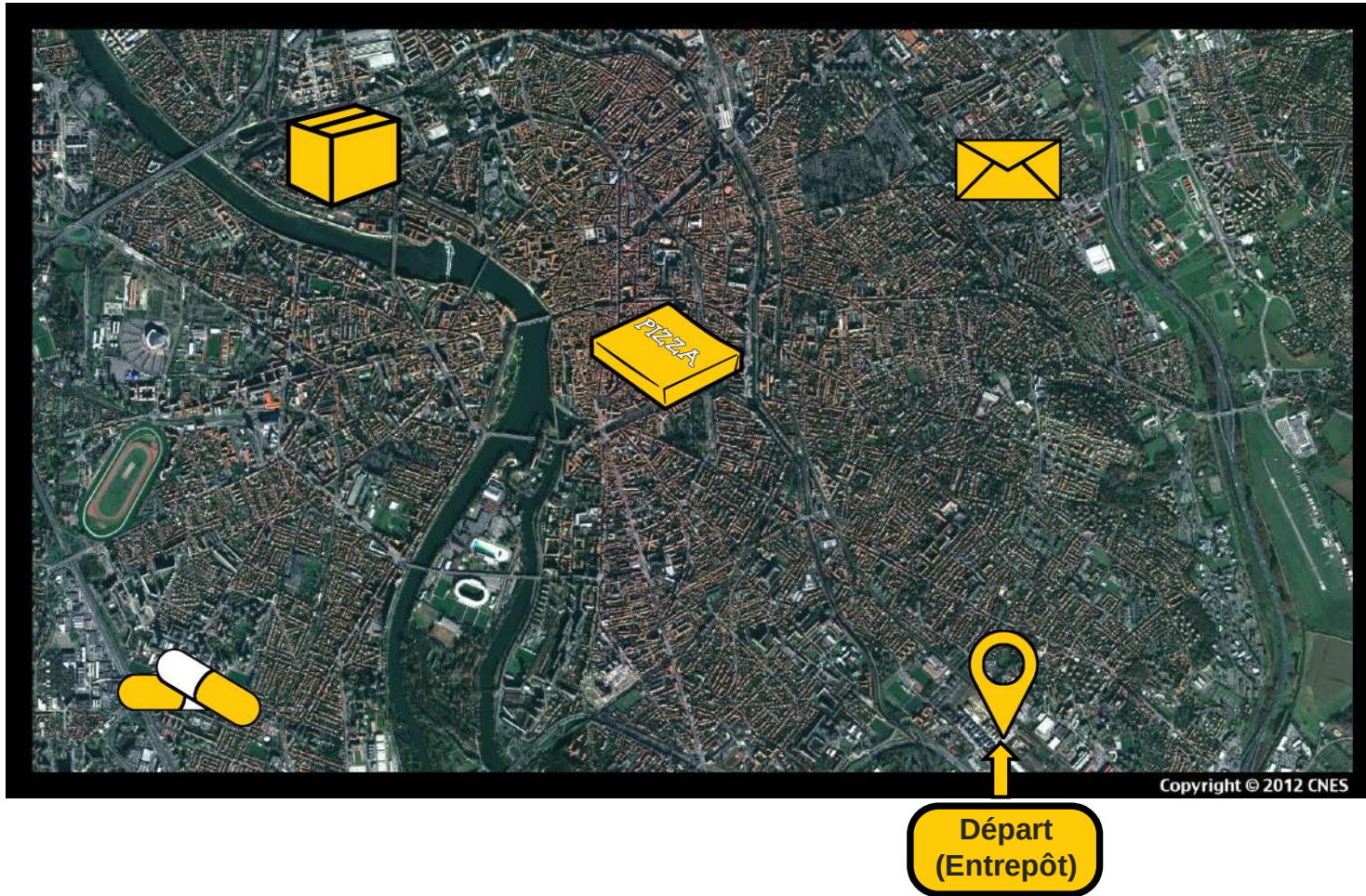
Contextualisation

- Essor livraison en ville
- Camion -> pollution, engorgement
- Piétonnisation des villes
- Législation



Exemple de drone

Contextualisation

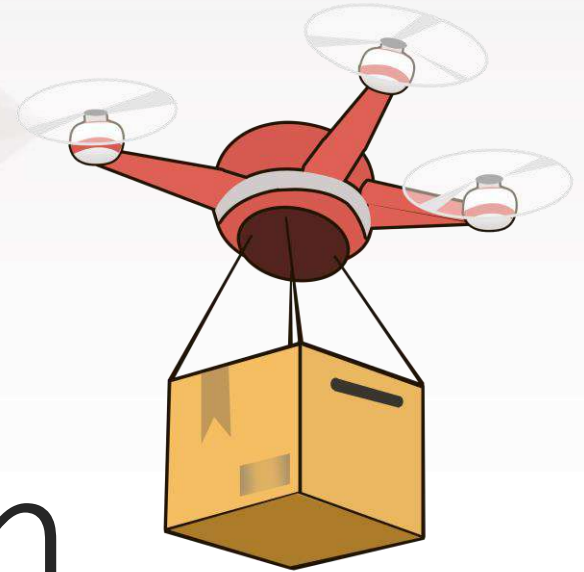


Exemple de casier

PLAN

- I. MODÉLISATION DE LA CONSOMMATION DU DRONE
- II. DÉCOMPOSITION DU PROBLÈME
- III. ALGORITHMES DE RECHERCHE D'UN CHEMIN OPTIMAL ÉNERGÉTIQUEMENT
- IV. EXPÉRIENCE ET ANALYSE DES RÉSULTATS

Modélisation de la consommation du drone

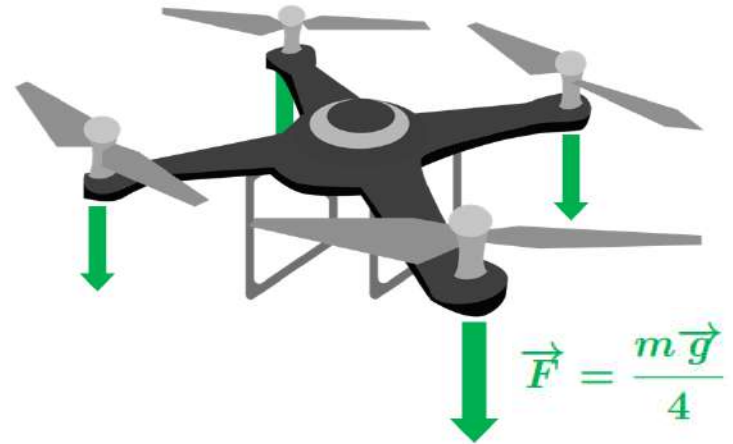


-
- Modèle de l'hélice parfaite de Rankine

Modèle de l'hélice de Rankine

- Poussée par hélice:

$$\vec{T} = -\frac{m \vec{g}}{4}$$



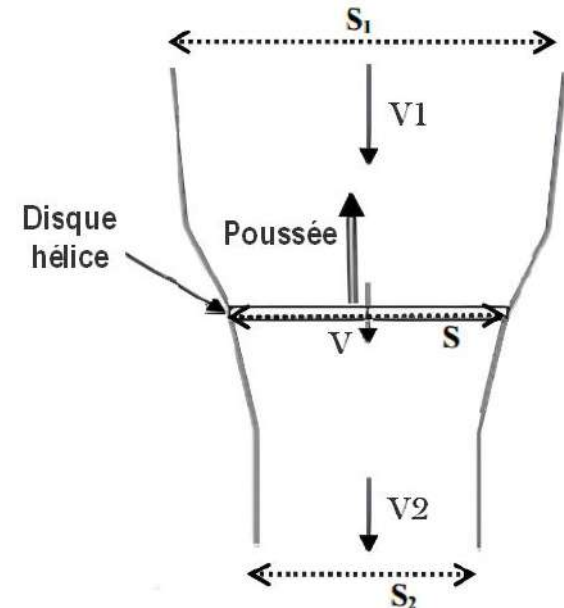
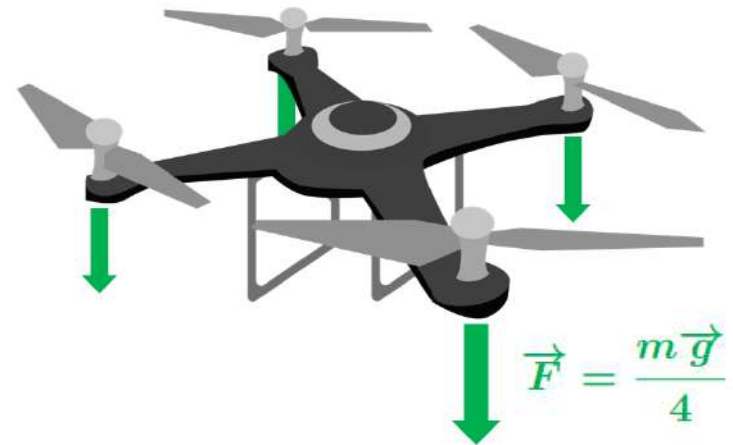
Modèle de l'hélice de Rankine

- Poussée par hélice:

$$\vec{T} = -\frac{m \vec{g}}{4}$$

- Hélice parfaite de Rankine:

$$\frac{T^3}{P^2} = 2\rho S = 2\pi\rho R^2$$



Modèle de l'hélice de Rankine

- Poussée par hélice:

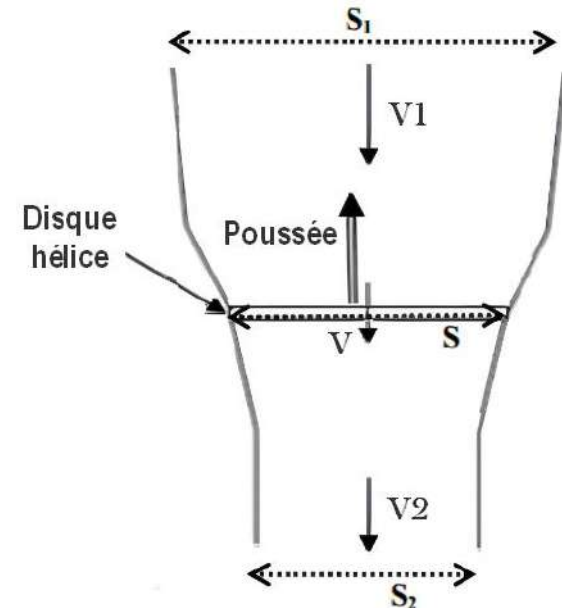
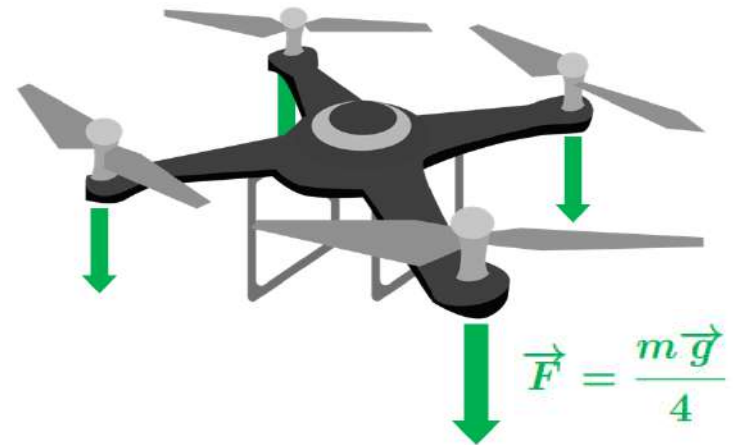
$$\vec{T} = -\frac{m \vec{g}}{4}$$

- Hélice parfaite de Rankine:

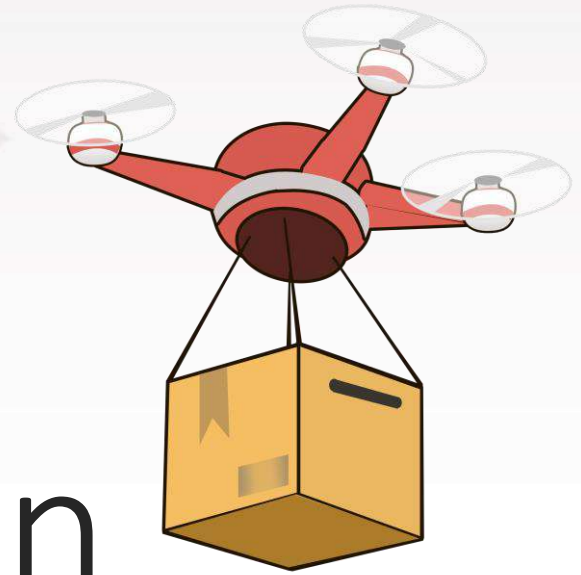
$$\frac{T^3}{P^2} = 2\rho S = 2\pi\rho R^2$$

- D'où:

$$P = \sqrt{\frac{\left(\frac{mg}{4}\right)^3}{2\pi\rho R^2}}$$



Décomposition du problème



- Principe d'optimalité de Bellman

Principe d'optimalité de Bellman



Copyright © 2012 CNES

Principe d'optimalité de Bellman



Copyright © 2012 CNES

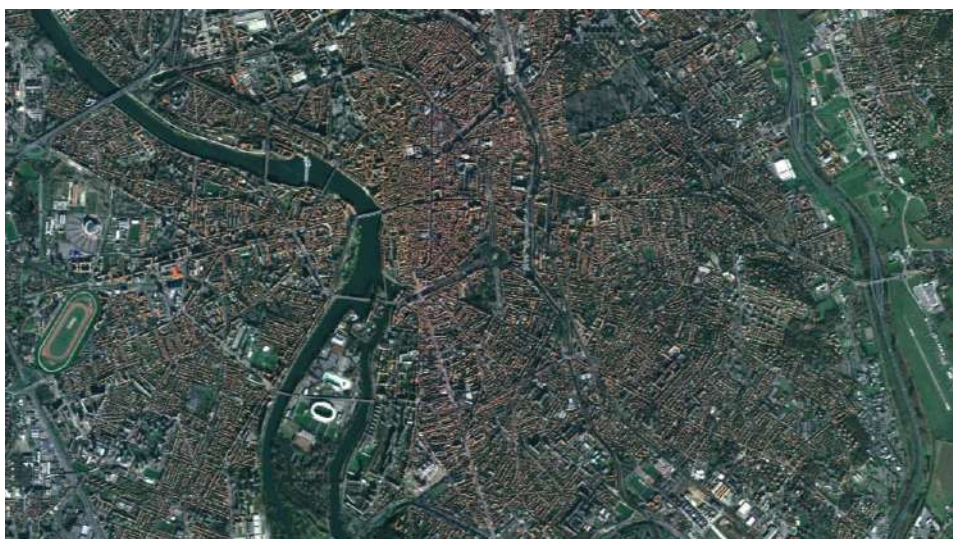
Algorithmes de recherche d'un chemin optimal



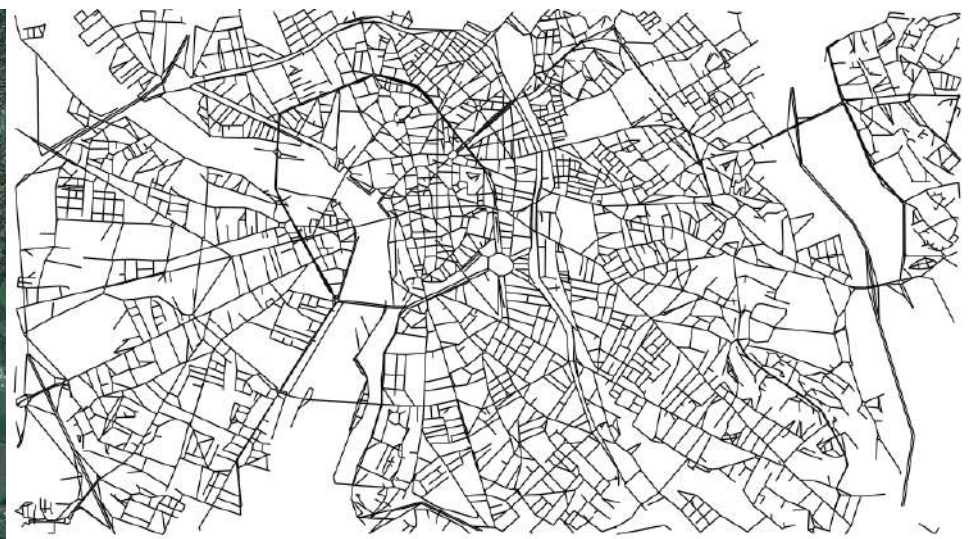
- i. Création du graphe d'une ville
- ii. Obtention du graphe complet (points à livrer)
- iii. Recherche du chemin de livraison optimal énergétiquement

Création du graphe de la ville

- Affichage du graphe avec le module NetworkX



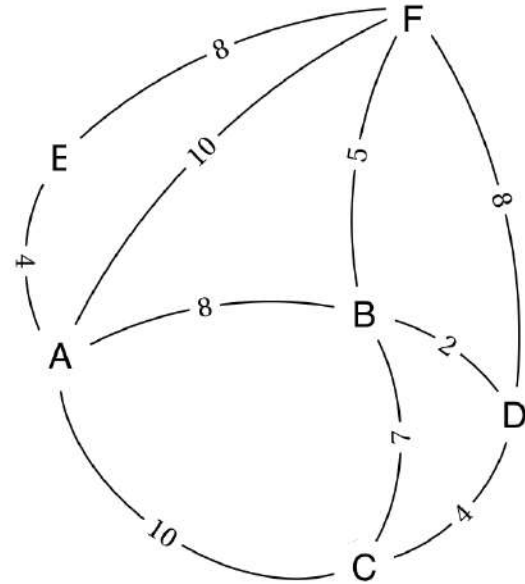
Vue satellite de Toulouse



Visualisation du graphe de Toulouse

Algorithme de Dijkstra

- Trouve le chemin le plus court entre un sommet donné et chacun des autres sommets



	A	B	C	D	E	F
Départ	∞	∞	0, C	∞	∞	∞
C	10, C	7, C		4, C	∞	∞
D	10, C	6, D			∞	12, D
B	10, C				∞	11, B
A					14, A	11, B
B					14, A	

Algorithme de Dijkstra

- Obtention du graphe complet



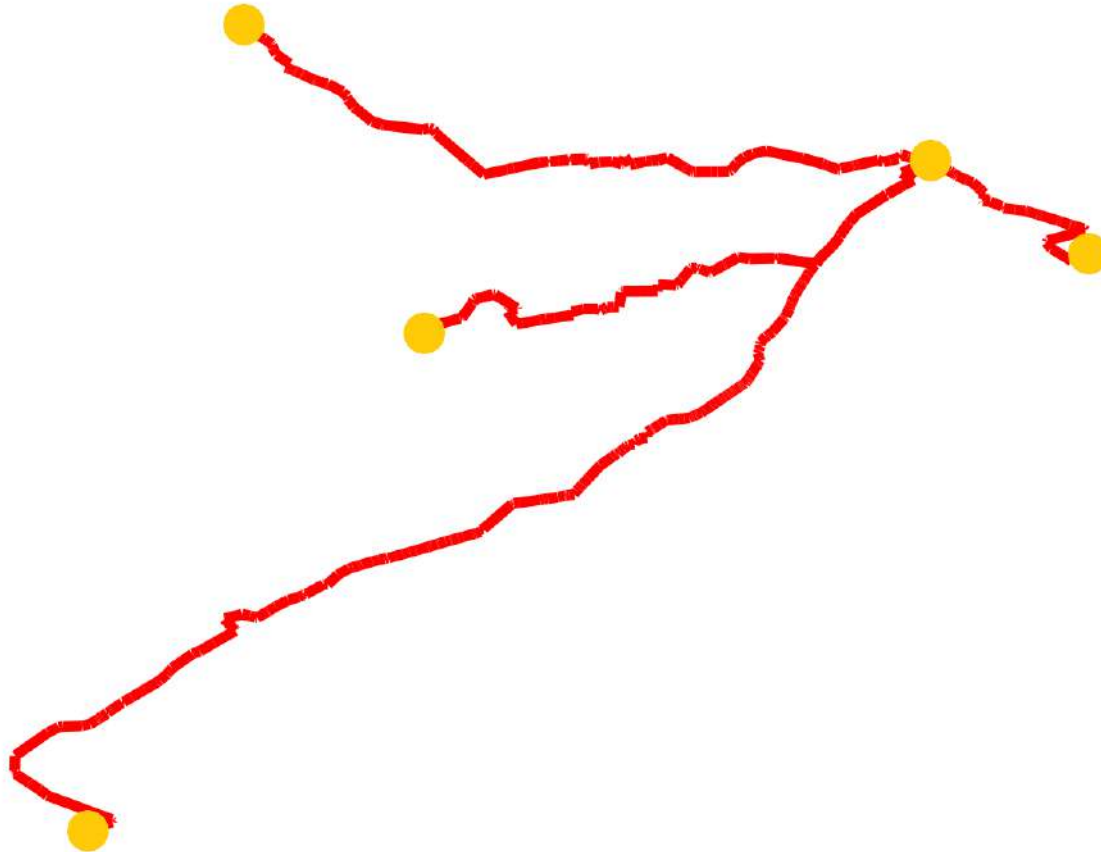
Algorithme de Dijkstra

- Obtention du graphe complet



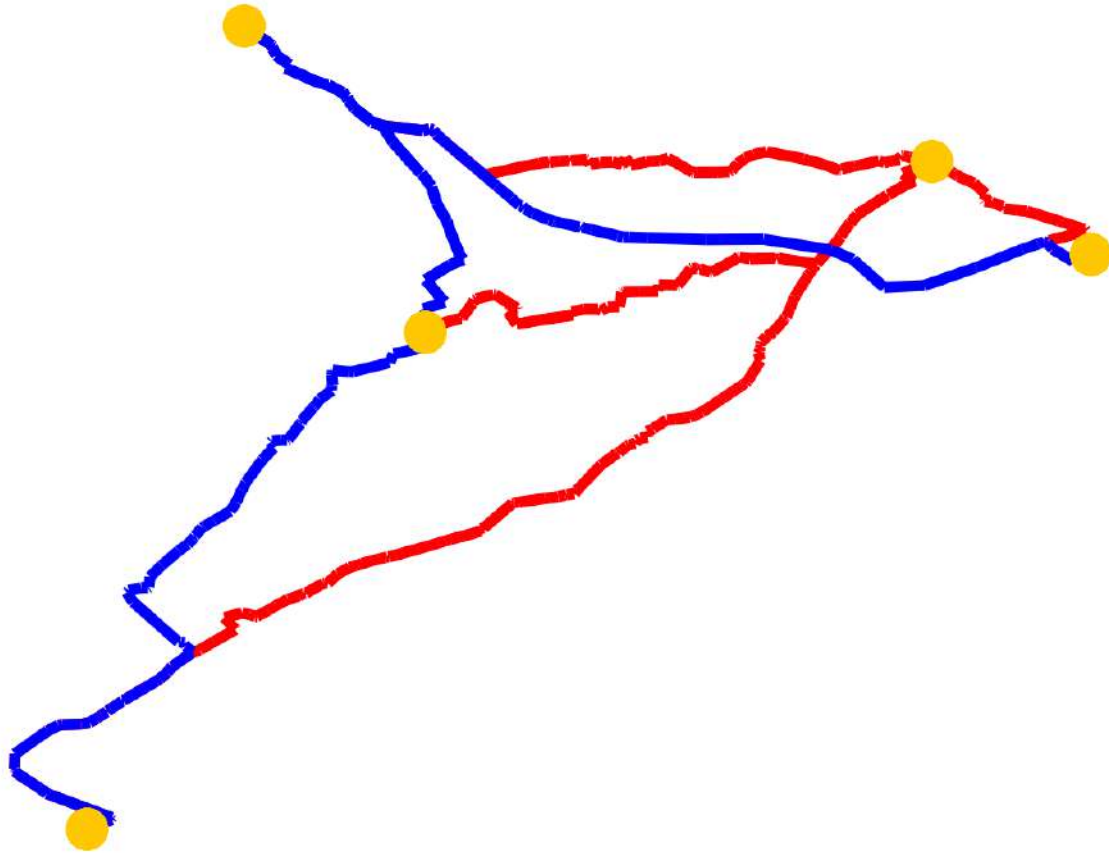
Algorithme de Dijkstra

- Obtention du graphe complet



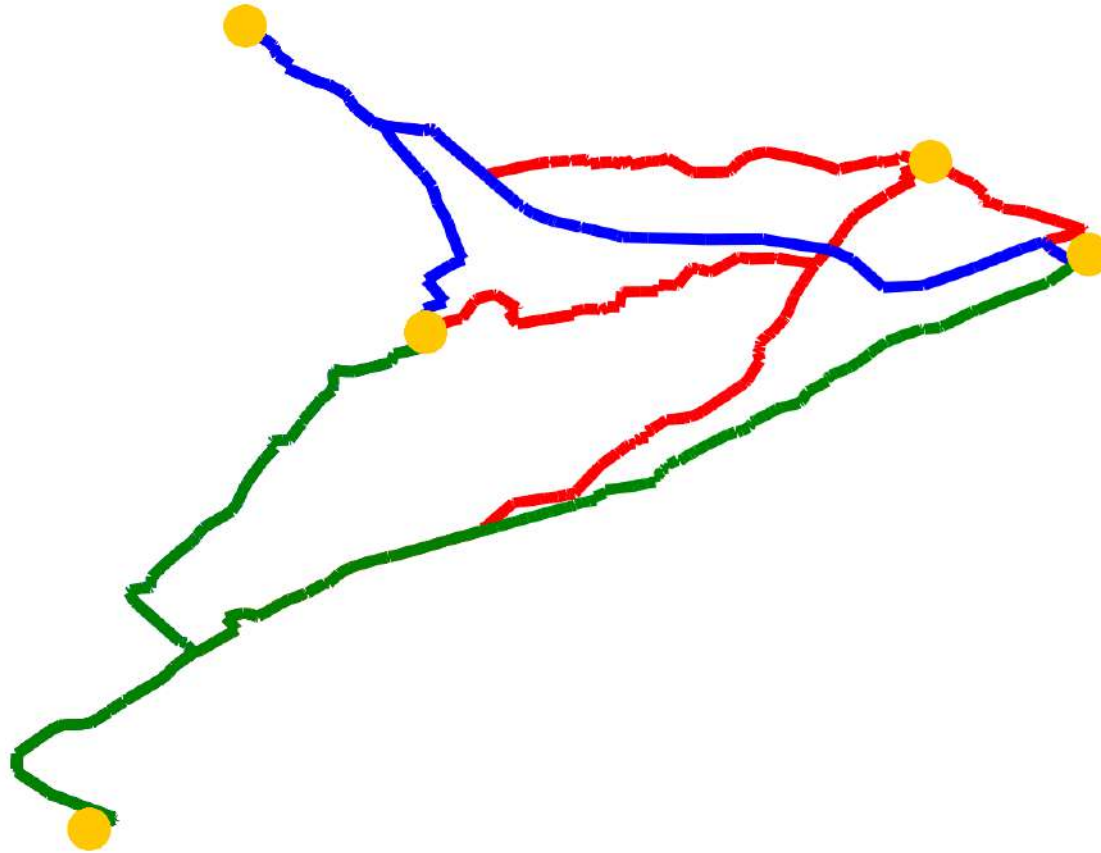
Algorithme de Dijkstra

- Obtention du graphe complet



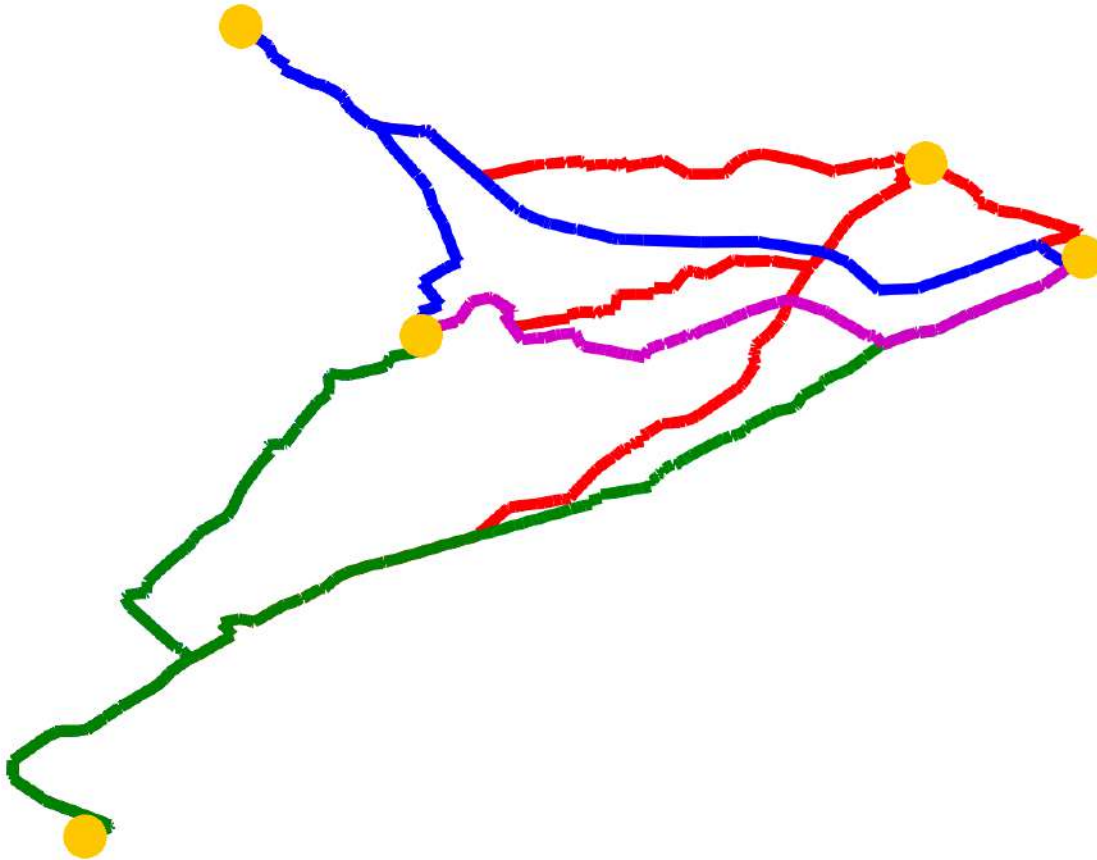
Algorithme de Dijkstra

- Obtention du graphe complet



Algorithme de Dijkstra

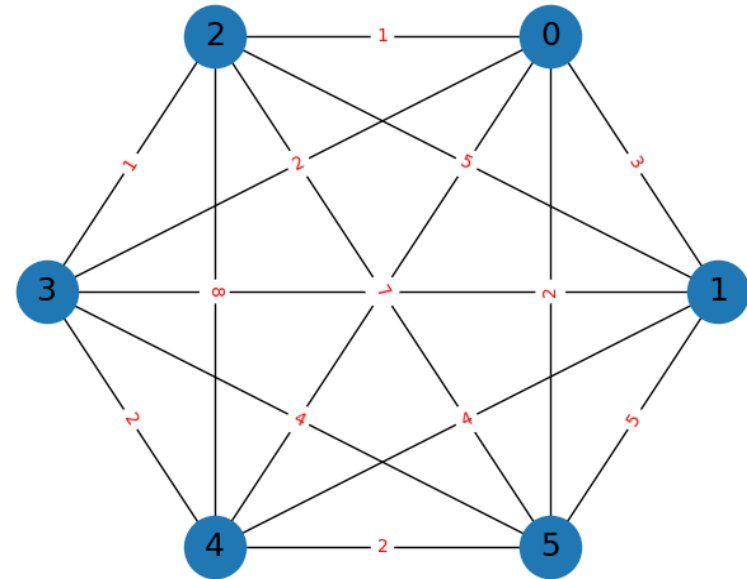
- Obtention du graphe complet



Recherche d'un chemin optimal

1ère approche:

- Calcul du coût énergétique de chaque chemin possible.
- Pour n sommets à livrer : $n!$ chemins possibles: inenvisageable pour $n > 12$.

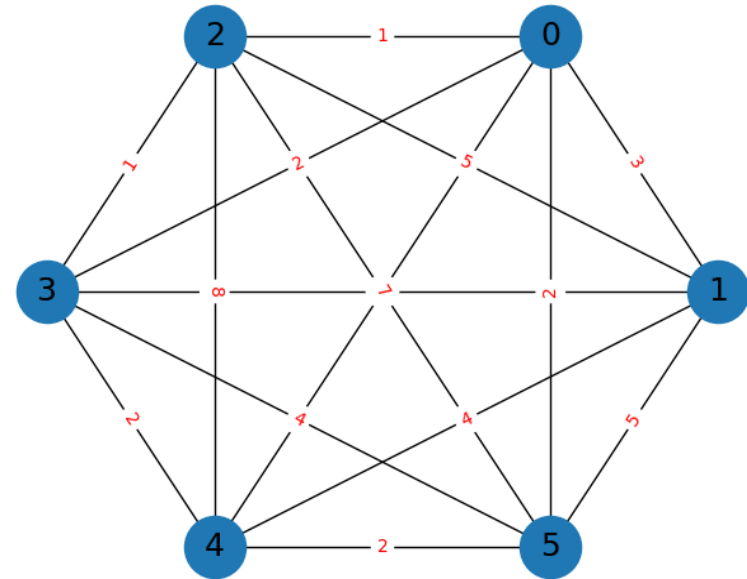


Graphe complet

Recherche d'un chemin optimal

1ère approche:

- Calcul du coût énergétique de chaque chemin possible.
- Pour n sommets à livrer : $n!$ chemins possibles: inenvisageable pour $n > 12$.
- Comparaison avec le plus court chemin :
Pour $n > 10$ colis,
surconsommation $< 5\%$ en moyenne.



Graphe complet

Recherche d'un chemin optimal

2^e approche:

- Pour une livraison de plus de 12 colis :
On cherche à approcher le plus court chemin.
- Problème du voyageur de commerce :
Heuristique de Christofides : Complexité en $O(n^3)$, le chemin obtenu est en moyenne 15% plus long que le chemin le plus court .

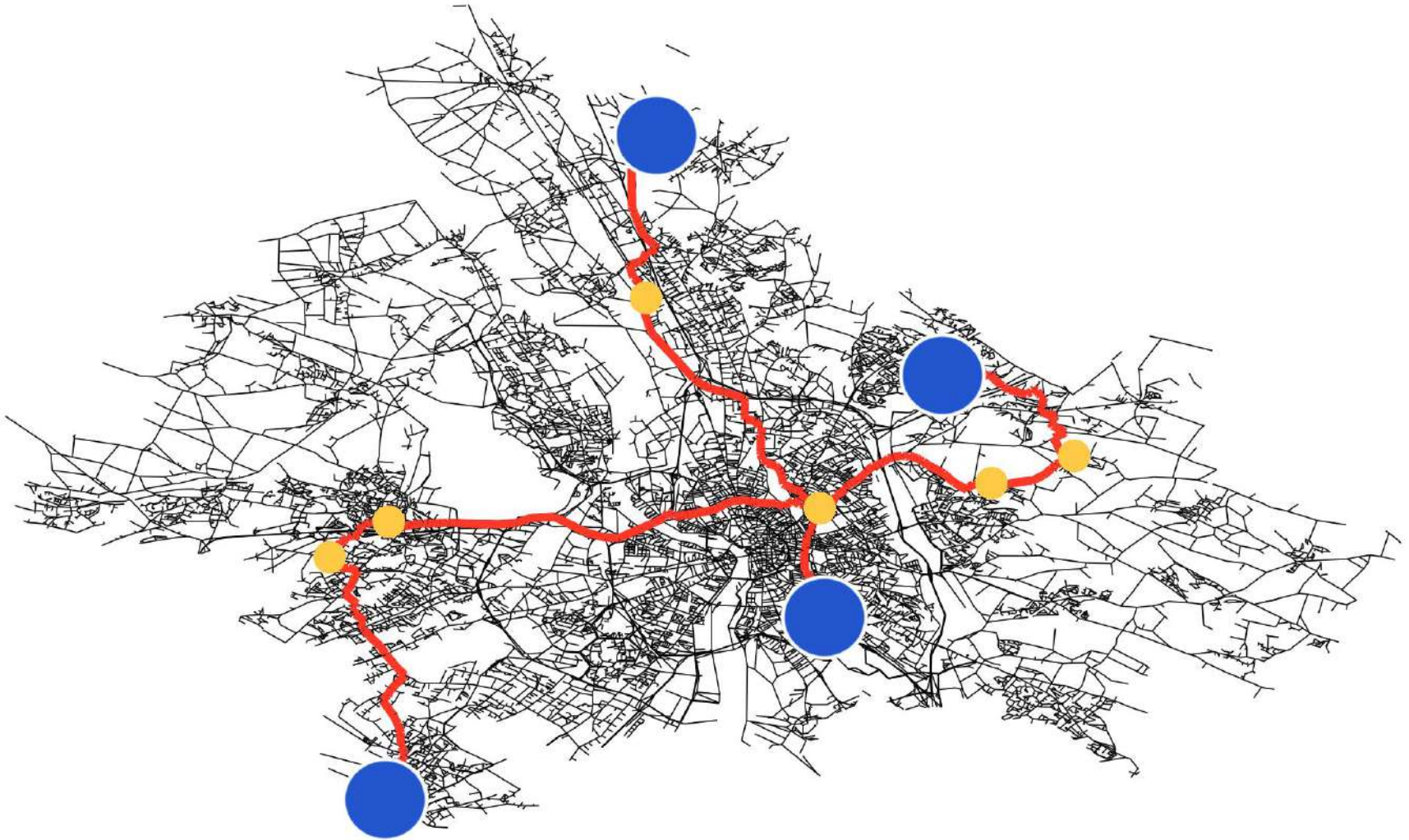
Heuristique de Christofides



Heuristique de Christofides



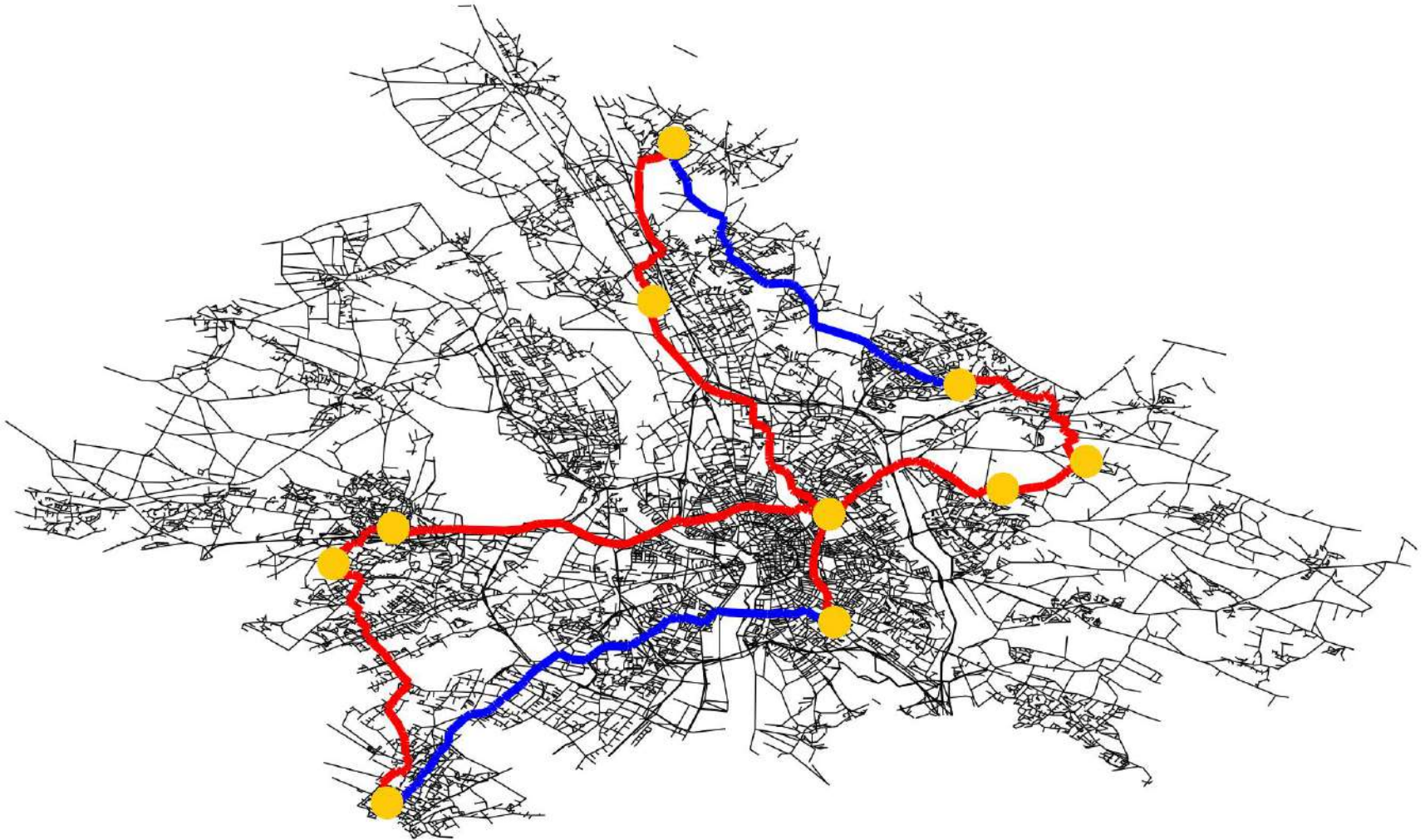
Heuristique de Christofides



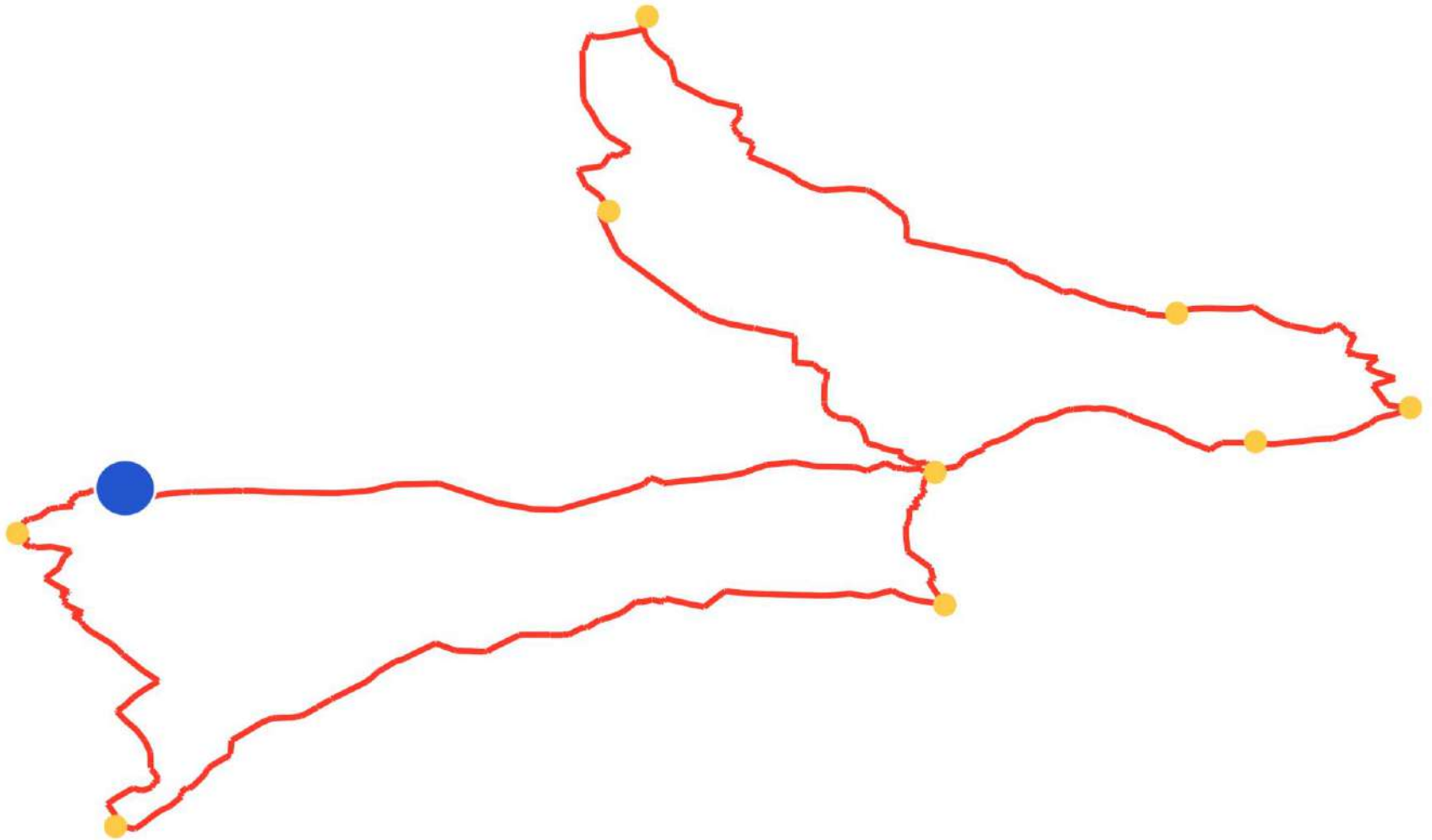
Heuristique de Christofides



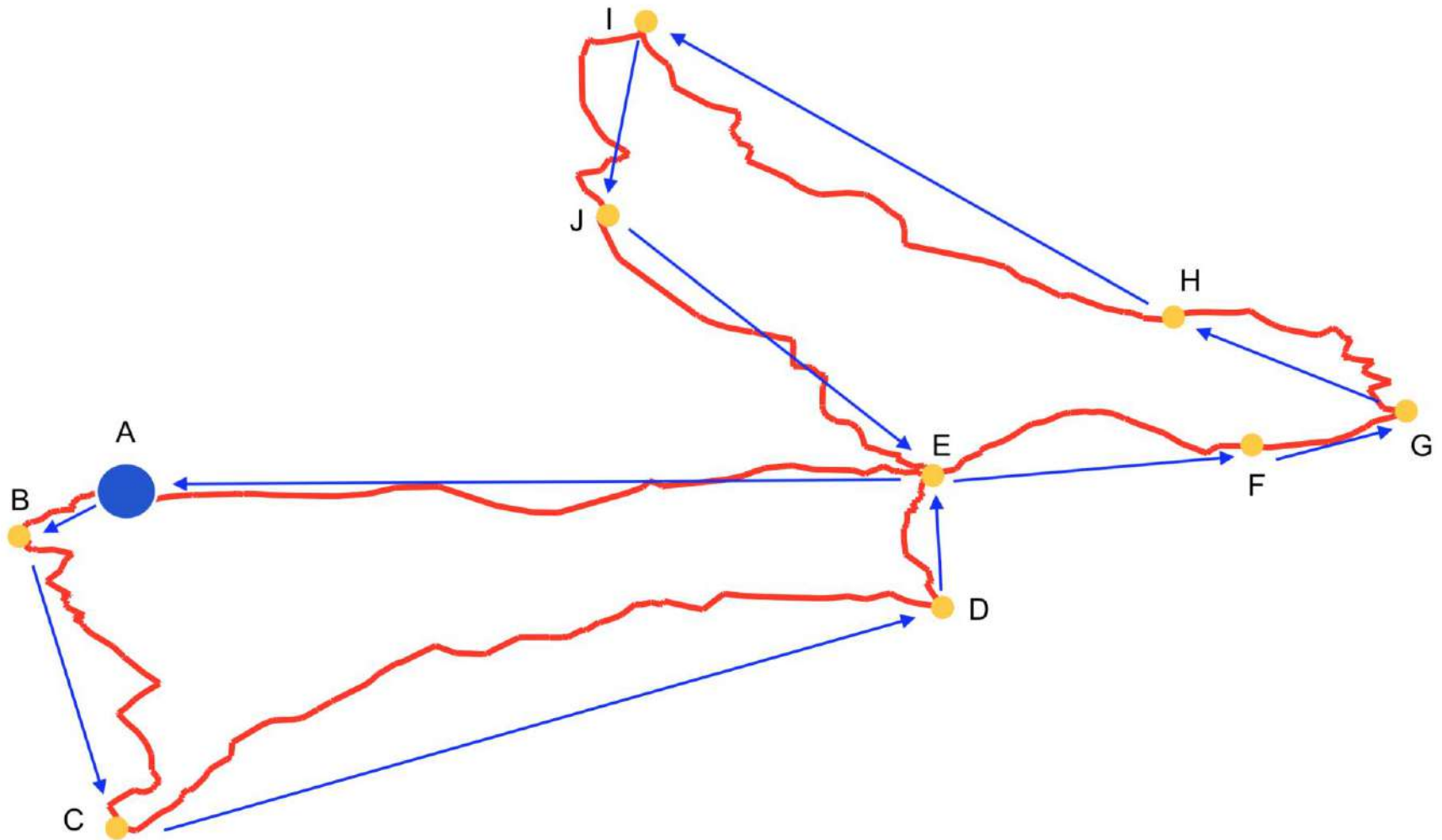
Heuristique de Christofides



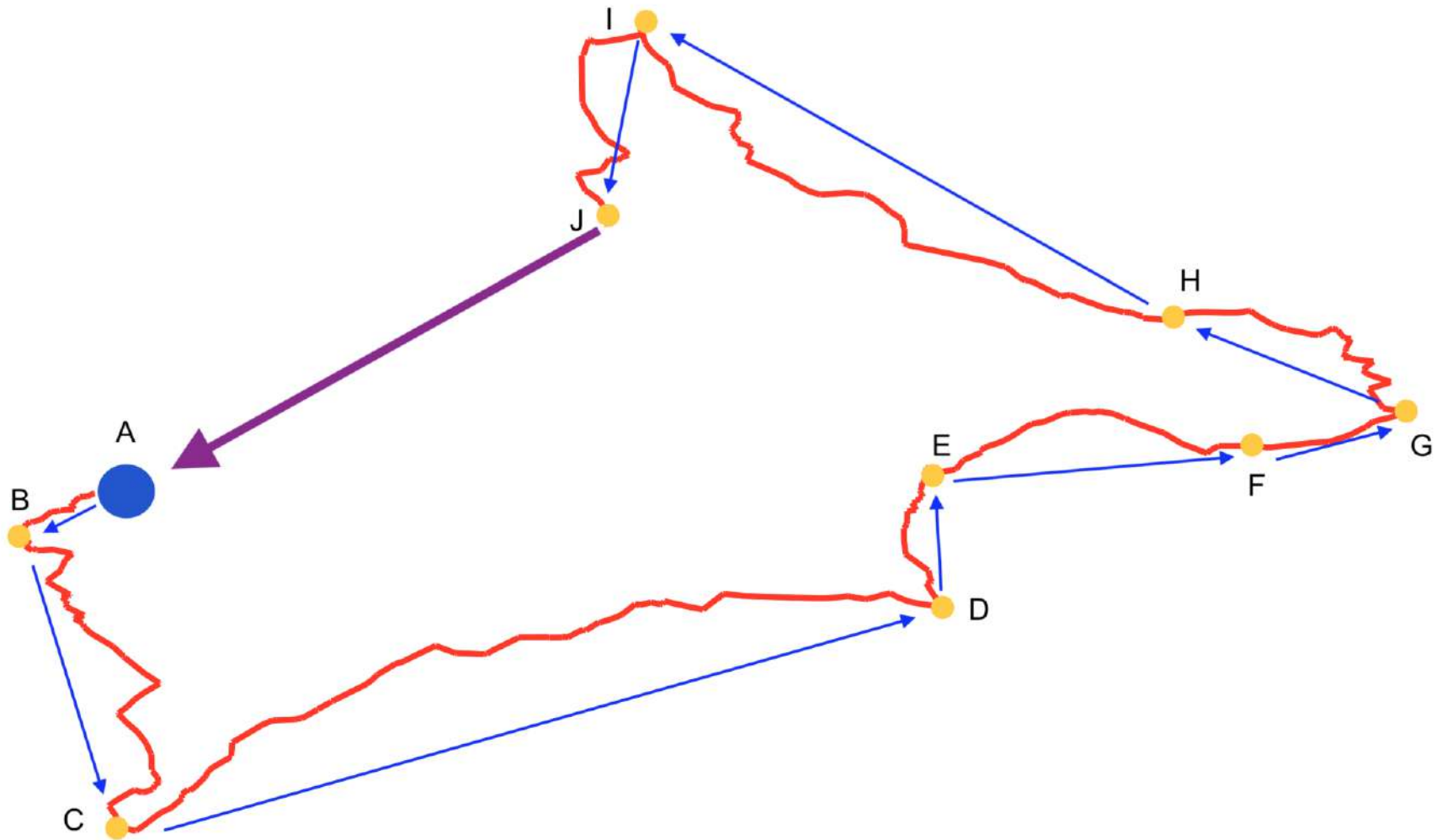
Heuristique de Christofides



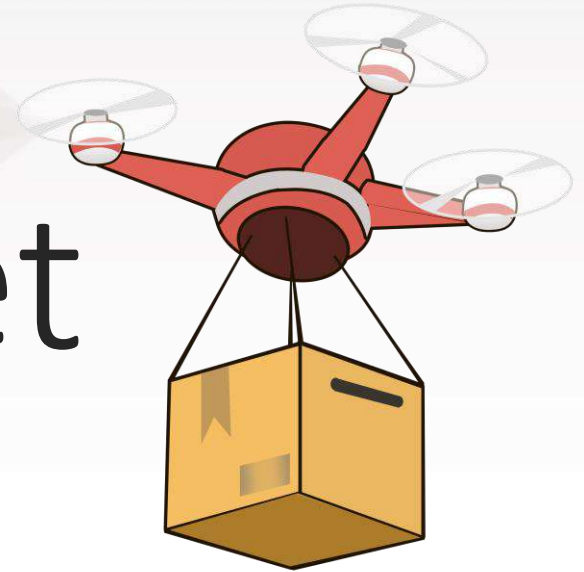
Heuristique de Christofides



Heuristique de Christofides



Expérience et Analyse des résultats



- i. Parcours du chemin optimal en conditions réelles
- ii. Comparaison avec un camion de livraison

Parcours du chemin optimal en conditions réelles

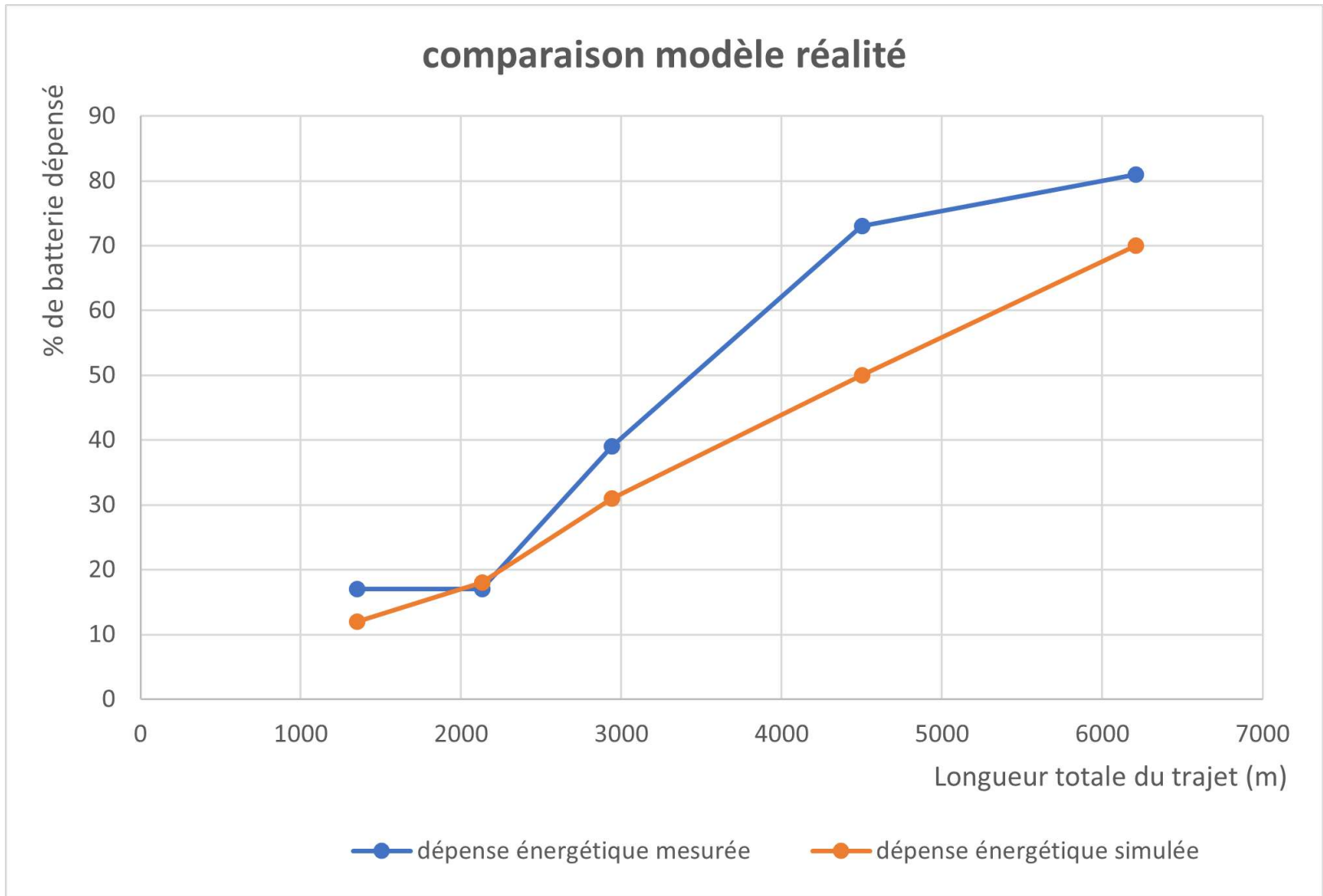


Drone portant un colis de 200g en vol stationnaire

Expérience :

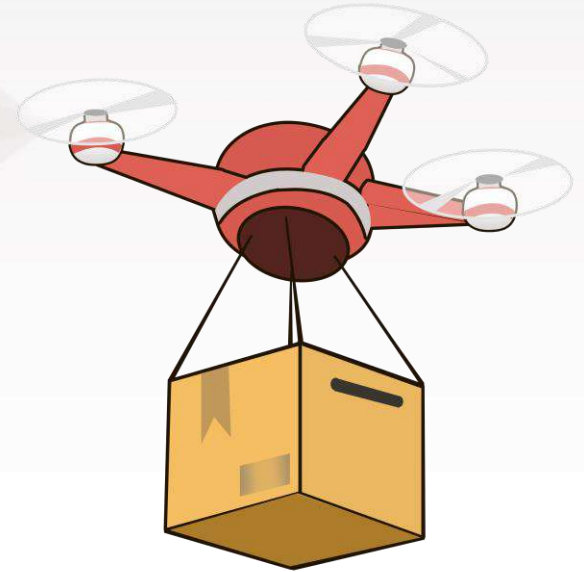
- 5 colis de masses différentes (20g, 40g, 60g, 80g et 120g)
- Placer des points dans un champ et référencer le graphe en python
- Simuler et mesurer la consommation pour chaque trajet

Parcours du chemin optimal en conditions réelles



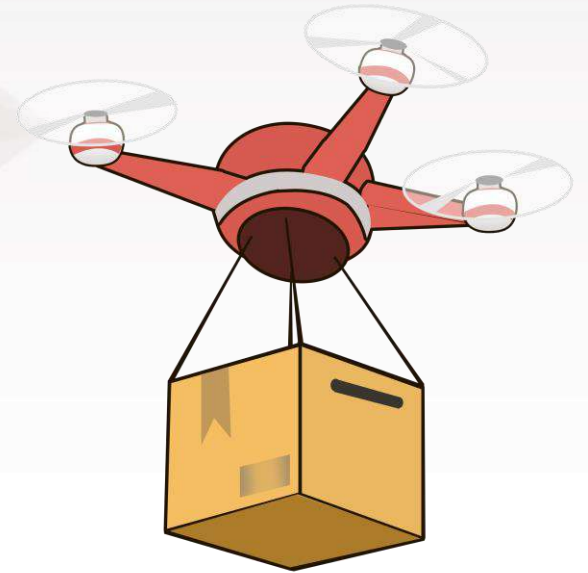
Comparaison avec un camion

	Drone	Camion de livraison
Nombre de colis livrés par heure	30	15 environ
Energie consommée par heure	1 kWh	20kWh

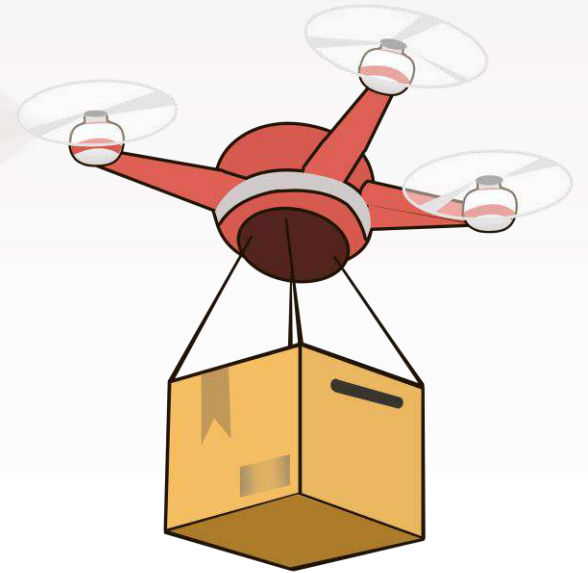


CONCLUSION

- i. Validation de l'étude : modèle physique, algorithmes
- ii. Limites de l'étude : météo, regroupement des livraisons par secteur, gestion du trafic aérien



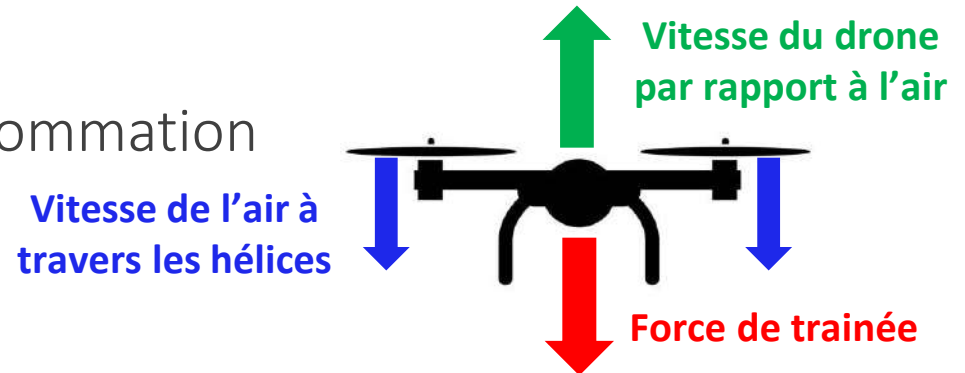
MERCI DE VOTRE ATTENTION



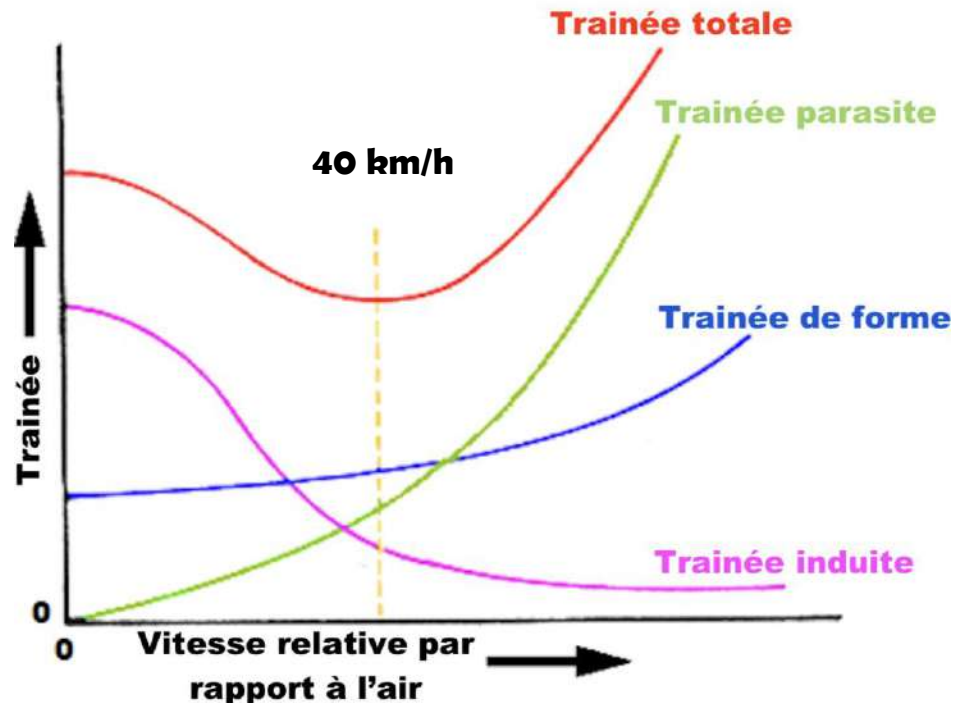
ANNEXES

Impact du vent

- Force de trainée → surconsommation



- Consommation $\simeq$ Consommation à l'arrêt
- Pour l'étude: $V = 40 \text{ km/h}$



Création du graphe de la ville

- Fichier au format csv (du site data.gouv.fr)

Excel :

1	Geo Point	Geo Shape	id_troncon	code_insee	rivoli	longueur	street
2	43.52484831	["coordinates":[[[1.367802798097895,43.525113420341384],[1.36856121872966,43.52458320368584]]], "type":"MultiLineString"]	14586	31588	3105880049W	85,0230991	IMP DE LAUB
3	43.63831456	["coordinates":[[[1.392094074728565,43.63879143546422],[1.39246558783388,43.63877655324973],[1.392445105517299,43.63850673491169],[1.392859804342123	20376	31069	3100690656W	177,318925	RUE MARCEL
4	43.55603504	["coordinates":[[[1.453175447138241,43.556219320282985],[1.453158482255509,43.5561423253095],[1.453134993781185,43.55601253882343],[1.453162392066121	39084	31555	3105552693N	95,4052683	CHEM DU DO
5	43.65894504	["coordinates":[[[1.471381622723704,43.65773500734892],[1.471374327242644,43.658238285936534],[1.471408974722148,43.65926509815929],[1.47138029926674	23339	31561	3105610220Z	265,405867	RUE DES ERAI
6	43.56999007	["coordinates":[[[1.595259386663668,43.56971962116817],[1.595100617059703,43.56984003728975],[1.594960517504552,43.56993999506369],[1.594699792959415	29836	31003	310003A010S	93,1740689	LOT LA CLE DI
7	43.57023458	["coordinates":[[[1.594267968837516,43.570309820145646],[1.594238552876329,43.57031029582065],[1.59417464221426,43.570299522132245],[1.59414812284197	38295	31003	310003A010S	34,2673134	LOT LA CLE DI
8	43.63901623	["coordinates":[[[1.38851155142965,43.63881543608739],[1.387996703365998,43.63921702587447]]], "type":"MultiLineString"]	16612	31069	3100690050D	60,9414906	CHE D AUSSO
9	43.57360621	["coordinates":[[[1.610805596727922,43.57361217651051],[1.610777261281027,43.573615168397204],[1.610754349044131,43.57361489289731],[1.61073016690096	38302	31163	3101630187U	9,95160854	AV DE LANTA
10	43.57353569	["coordinates":[[[1.610697781925525,43.57358311213633],[1.610691543468572,43.57355526913917],[1.610708871286096,43.57353215117265],[1.610730665079521	38303	31163	3101630187U	11,386634	AV DE LANTA
11	43.58551052	["coordinates":[[[1.585637070096106,43.58566852253657],[1.585621186157055,43.58559238787439],[1.585422083834038,43.585414457680855],[1.58534656584614	38306	31163	3101630014F	40,1547092	RUE DE LAUF
12	43.61152639	["coordinates":[[[1.512660644893451,43.61069614095329],[1.512906244589587,43.610884142486526],[1.512978981695052,43.61099252467799],[1.51315710297817	25950	31044	3100440886W	209,070208	RUE VALENTI
13	43.59614823	["coordinates":[[[1.608417406066536,43.59497923513869],[1.608417406066536,43.59497923513869],[1.60859952521505,43.59528026555597],[1.608659349699811	30982	31163	3101630060F	267,325217	RUE DE LA CC
14	43.54506057	["coordinates":[[[1.456716994484685,43.54432348613557],[1.456698504973609,43.54460717466771],[1.45679524193661,43.5450166557063],[1.456856152146732,4	7081	31555	3105553089U	363,602513	RUE DE L ESTI
15	43.66844014	["coordinates":[[[1.379412281475251,43.66844683064286],[1.379534892152509,43.668428762530034],[1.379650508139558,43.66845643408044]]], "type":"MultiLine	13740	31056	3100560092F	19,9084696	PL DE LA MAIL
16	43.55218787	["coordinates":[[[1.533684470259324,43.5525619604085],[1.534166433130342,43.55181378671421]]], "type":"MultiLineString"]	11625	31506	3105060670C	91,7924077	AV DE GAMEV
17	43.59869042	["coordinates":[[[1.325182237866336,43.59873062287445],[1.325167741873849,43.59875284012585],[1.325066969287807,43.59877044196066],[1.32503334843011,	38308	31149	3101490756Y	54,3553159	ALL DU LAVAI
18	43.63187650	["coordinates":[[[1.45908476222522,43.631696729461694],[1.459330872820541,43.632056273422265]]], "type":"MultiLineString"]	687	31555	3105555028B	44,6110362	CHE DE LAPU.
19	43.59830418	["coordinates":[[[1.606005515514372,43.59817579526299],[1.606005515514372,43.59817579526299],[1.605890553086381,43.59834835558598],[1.605784952830252	30086	31163	3101630013E	31,9283044	PL DES ARCHE

- Sélection des données utiles

Python: [coordonnées départ, coordonnées arrivée, longueur]

Tronçon 1: [[1.367802798097895, 43.525113420341384], [1.36856121872966, 43.52458320368584], 85.0230991357258]
Tronçon 2: [[1.392094074728565, 43.63879143546422], [1.392819412379763, 43.637573174449821], 177.318925208934]
Tronçon 3: [[1.453175447138241, 43.556219320282985], [1.453175447138241, 43.556219320282985], 95.4052683349242]

⋮

⋮

⋮

⋮

Tronçon 33096: [[1.285883693900528, 43.62528097769707], [1.285715623685736, 43.624639439347455], 74.9544412409529]

Modélisation Physique

Hypothèses du modèle de l'hélice de Rankine :

- *Fluide (air) incompressible : $\rho = cte$*
- *Régime permanent : $q_m = cte$*
- *Disque hélice infiniment fin, composé d'une infinité de pales*
- *Fluide non visqueux*

Théorème de Bernoulli :

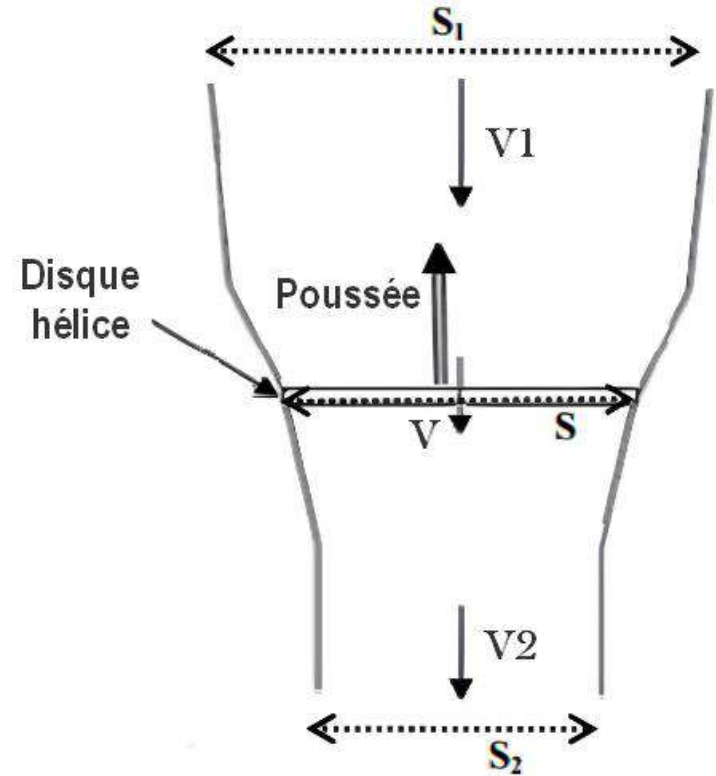
$$\frac{\rho V^2}{2} + \rho g z + \text{Pression} = cte$$

le long d'une ligne de courant

Théorème d'Euler :

On applique le PFD à un cylindre de section S élémentaire de fluide

$$\begin{aligned} \text{autour du plan hélice : } \frac{d\vec{p}}{dt} &= \vec{F}_{\text{hélice} \rightarrow \text{fluide}} = \frac{\vec{p}(t+dt) - \vec{p}(t)}{dt} = \frac{m\vec{v}_2 - m\vec{v}_1}{dt} \\ &= q_m(\vec{v}_2 - \vec{v}_1) \end{aligned}$$



Modélisation Physique

Etablissement de l'équation de la puissance :

V_1 = Vitesse de l'air à l'infini amont qui vaut 0 en vol stationnaire

V_2 = Vitesse de l'air à l'infini aval

Le théorème d'Euler donne : $F = q V_2 = \rho S_2 V_2^2$

D'autre part, l'application du PFD et du théorème de Bernoulli aux deux

faces du plan hélice donne : $F = S \Delta P = S \rho \frac{V_2^2}{2}$

Donc $S_2 = \frac{S}{2}$, et la conservation du débit massique donne $SV = S_2 V_2$ d'où

$V_2 = 2V$ et $F = \rho S_2 V V_2 = 2\rho S V^2$

La puissance fournie par l'hélice doit être égale à l'énergie cinétique acquise par l'air lors de la traversée du rotor :

$$P = \frac{1}{2} q V_2^2 = \frac{1}{2} \rho S V V_2^2 = 2\rho S V^3$$

$$\text{D'où } \frac{P^3}{F^2} = 2\rho S = \text{Cte} \quad \text{et} \quad P = \sqrt{\frac{\left(\frac{mg}{4}\right)^3}{2\pi\rho R^2}}$$

Démonstration Christofides

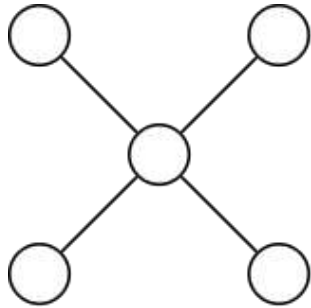
$$\begin{aligned}\sum \text{degrés des sommets} &= \sum \text{degrés impairs} + \sum \text{degrés pairs} \\ &= 2 \times \text{nombre d'arcs}\end{aligned}$$

↘ pair

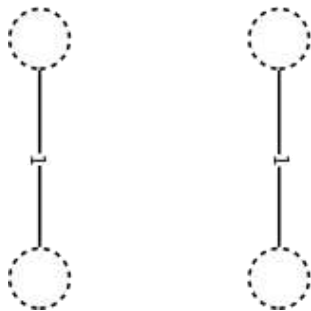
↘ pair

$\Rightarrow \sum \text{degrés impairs}$ est paire $\Rightarrow$ nombre pair de sommets de degré impair

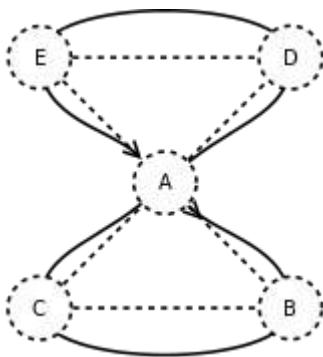
Démonstration Christofides




 poids total arbre couvrant
de poids minimal $\leq$ longueur plus
court trajet




 poids total couplage
parfait de poids minimal $\leq \frac{\text{longueur plus court trajet}}{2}$




 poids total cycle eulérien $\leq \frac{3}{2}$ longueur plus
court trajet

Bellman

```
213 G2=nx.Graph()
214 C2=[]
215 L2=[]
216 for i in tqdm(C): # C= Liste des coordonnées de toutes les intersections de Toulouse
217     if i[0] > 1.399145 and i[0] < 1.503258 and i[1] > 43.573202 and i[1] < 43.617040:
218         C2.append(i)
219         L2.append(C.index(i))
220 G2.add_nodes_from(L2)
221 posito3={}
222 for i in L2:
223     posito3[i]=C[i]
224 rows, cols = M.nonzero() # M= Matrice d'adjacence de Toulouse (Matrice creuse)
225 for i, j in zip(rows, cols):
226     if i <= j:
227         if i in L2 and j in L2:
228             G2.add_edge(i, j, weight=M[i, j])
229 labels_edges = {}
230 labels_edges = {edge: G2.edges[edge]["weight"] for edge in G2.edges}
231
232 bg=[1.399145, 43.573202] # Coins du rectangle de la photo
233 hd=[1.503258, 43.617040]
234 hg=[1.399145, 43.617040]
235 bd=[1.503258, 43.573202]
236
237 colis=['Départ', '5 kg', '0.5 kg', '0.1 kg', '1 kg']
238 J=nx.Graph()
239 J.add_nodes_from(colis)
240 posito={}
241 coord=[[1.475334438274966, 43.57870959790727], [1.426947462227057, 43.60792337544156],
242         [1.447025719178776, 43.596436979947015], [1.478582454254292, 43.605983760146], [1.414944282469067,
243         43.583407610802695]] # Coordonnées sommets à livrer
244 for i in colis:
245     posito[i]=coord[colis.index(i)]
246 plt.clf()
247 nx.draw(J, posito, with_labels=True, node_size=1800, node_color='FFCA08') #Tracé des sommets (masses)
248 nx.draw_networkx_edges(G2, posito3, width=0.9) # Tracé des routes de Toulouse
249
250 L=[C.index(i) for i in coord]
251 B=completion(M,L) # Tracé du plus court chemin pour chaque couple
252
253 ##
254 Colis=[0,5,0.5,0.1,1]
255 CH0=chemin_opti(B,Colis)
256 ChemOpti=CH0[0]
257
258 S=nx.Graph()
259 S.add_nodes_from([k for k in range(1,len(B))])
260 S.add_node('Départ')
261 position={}
262 position['Départ']=[1.475334438274966, 43.57870959790727]
263 for i in range(1,len(B)):
264     position[i]= coord[ChemOpti[i]]
265
266 nx.draw(S, position, node_size=1800, with_labels=True, node_color='FFCA08') # Tracé de l'ordre optimal
267 plt.show()
```


Obtention points et graphe ville

```
1 import json
2 import matplotlib.pyplot as plt
3 import networkx as nx
4 import numpy as np
5 import pandas as pd
6 from scipy.sparse import lil_matrix
7 from tqdm import tqdm
8
9 ## Convertir Base de donnée en matrice d'adjacence
10 data = pd.read_csv(r" chemin d'accès du fichier ", sep=";")
11 T = pd.DataFrame(data, columns=["Geo Shape", "longueur"])
12
13 # Supprimer les lignes avec une valeur NaN dans la colonne 'Geo Shape'
14 T.dropna(subset=["Geo Shape"], inplace=True)
15
16 # Conversion des chaines de caractère JSON en dictionnaires
17 T["Geo Shape"] = T["Geo Shape"].apply(json.loads)
18 # geo shape: dictionnaire avec clé ['coordonnées', 'type'].
19 # 'type' ne nous intéresse pas donc on ne garde que les coordonnées
20 T["Geo Shape"] = T["Geo Shape"].apply(
21     lambda x: x["coordinates"][0]
22     if isinstance(x, dict) and "coordinates" in x
23     else None)
24
25 # Conversion de la colonne "Geo Shape" en un array numpy
26 geo_shapes = np.array(T["Geo Shape"])
27
28 # récupérer la première et la dernière coordonnée de chaque ligne
29 first_coords = [shape[0] for shape in geo_shapes]
30 last_coords = [shape[-1] for shape in geo_shapes]
31
```

Obtention points et graphe ville

```
35 # Création du tableau [coord départ, coord arrivée, longueur chemin] ... ]  
    pour chaque sommet  
36 T = [[first, last, length]  
37     for first, last, length in zip(first_coords, last_coords, T["longueur"])]  
38  
39 C = [] # Juste les coordonnées des sommets  
40 coordinate_set = set()  
41  
42 for line in T:  
43     start_coord, end_coord, length = line[0], line[1], line[2]  
44  
45     start_coord_tuple = tuple(start_coord)  
46     if start_coord_tuple not in coordinate_set:  
47         C.append(start_coord)  
48         coordinate_set.add(start_coord_tuple)  
49  
50     end_coord_tuple = tuple(end_coord)  
51     if end_coord_tuple not in coordinate_set:  
52         C.append(end_coord)  
53         coordinate_set.add(end_coord_tuple)  
54  
55 n = len(C)  
56  
57 # Création de la matrice d'adjacence M  
58 M = lil_matrix((n, n))  
59  
60 coord_to_index = {tuple(coord): index for index, coord in enumerate(C)}  
61  
62 for i in T:  
63     a = coord_to_index[tuple(i[0])]  
64     b = coord_to_index[tuple(i[1])]  
65     if a != b:  
66         M[a, b] = i[2]  
67         M[b, a] = i[2]
```

Obtention points et graphe ville

```
69
70 # Affichage
71 P = [k for k in range(n)]
72 G = nx.Graph()
73 G.add_nodes_from(P)
74 D = []
75 rows, cols = M.nonzero()
76 for i, j in zip(rows, cols):
77     if i <= j:
78         G.add_edge(i, j, weight=M[i, j])
79
80 labels_edges = {}
81 labels_edges = {edge: G.edges[edge]["weight"] for edge in G.edges}
82 posi = {}
83 for k in P:
84     posi[k] = C[k]
85
86 nx.draw_networkx_edges(G, posi,width=0.7)
87 plt.show()
88
```

Algorithme de Dijkstra

```
1 import tqdm as tqdm
2 import networkx as nx
3 import matplotlib.pyplot as plt
4 ##
5
6 #acquiert les plus courts chemins et les trajets entre un sommet de départ et
7 #tous les autres sommets donnés
8 def dijkstra(D,dep,L): # (Dictionnaire, point de départ, liste de sommets)
9     assert all(D[u][v] >= 0 for u in D.keys() for v in D[u].keys())
10    precedent = {i:None for i in D.keys()} # Dictionnaire des
11    antécédents
12    dejaTraite = {i:False for i in D.keys()} # Dictionnaire de
13    connaissance ou non des chemins les plus courts
14    distance = {i:float('inf') for i in D.keys()}# Dictionnaire de la
15    longueur des chemins
16    distance[dep] = 0
17    a_traiter = [(0, dep)]
18    while False in [dejaTraite[i] for i in L]:
19        dist_noeud, noeud = a_traiter.pop()
20        if not dejaTraite[noeud]:
21            dejaTraite[noeud] = True
22            for voisin in D[noeud].keys():
23                dist_voisin = dist_noeud + D[noeud][voisin]
24                if dist_voisin < distance[voisin]:
25                    distance[voisin] = dist_voisin
26                    precedent[voisin] = noeud
27                    a_traiter.append((dist_voisin, voisin))
28    a_traiter.sort(reverse=True)
29    return [distance, precedent]
```

Algorithme de Dijkstra

```
27 #transforme Matrice en dictionnaire
28 def dictionnaire(M):          # (matrice d'adjacence (sparse matrice))
29     D = {}                    # Dictionnaire d'adjacence
30     rows, cols = M.nonzero()
31     L = [[] for _ in range(M.shape[0])]
32     for i,j in zip(rows, cols):
33         L[i].append(j)
34     for i in range(M.shape[0]):
35         d = {j:M[i,j] for j in L[i]}
36         D[i] = d
37     return D
38
```


Algorithme de Dijkstra

```
39 #acquiert le poids des plus courts chemins et les trajets entre chaque couple
    de sommets à livrer
40 def completion(M,L): # (matrice d'adjacence(sparse matrice), Liste de sommets)
41     D = dictionnaire(M)
42     M2 = [[0 for i in range(len(L))] for j in range(len(L))] #Mat d'adjacence
43     M3 = [[[ ] for i in range(len(L))] for j in range(len(L))] #Mat des chemins
44     Poids = []
45     Chemins = []
46     '''nx.draw_networkx_edges(G,posi,width=0.6)'''
47     for i in tqdm(range(len(L))):
48         A = dijkstra(D,L[i],L)
49         Poids.append(A[0])
50         Chemins.append(A[1])
51     for i in range(len(L)):
52         for j in range(i,len(L)):
53             M2[i][j] = Poids[i][L[j]]
54     for i in range(len(L)):
55         for j in range(i,len(L)):
56             M2[j][i] = M2[i][j]
57     for i in range(len(L)):
58         for j in range(i+1,len(L)):
59             v = L[j]
60             while v != None:
61                 if Chemins[i][v] != None:
62                     M3[i][j].append(Chemins[i][v])
63                     v = Chemins[i][v]
64             M3[i][j].reverse()
65             M3[i][j].append(L[j])
```

Algorithme de Dijkstra

```
66     '''Chemin=M3[i][j]
67     Gr=nx.Graph()
68     Gr.add_nodes_from(Chemin)
69     position={}
70     for l in Chemin:
71         position[l] = C[l]
72     E=[]
73     for l in range(len(Chemin)-1):
74         E.append((Chemin[l],Chemin[l+1],M[Chemin[l],Chemin[l+1]]))
75     Gr.add_weighted_edges_from(E)
76     labels = {}
77     labels = {edge:Gr.edges[edge]['weight'] for edge in Gr.edges}
78     S=nx.Graph()
79     S.add_nodes_from(L)
80     position2 = {}
81     for l in L:
82         position2[l] = C[l]
83     nx.draw(S,position2,node_size=200)
84     nx.draw_networkx_edges(Gr,position,edge_color='r',width=2)
85     plt.show()'''
86     return M2, M3
87
```

Recherche chemin optimal

```
49 Vitesse_moy = 10 # m/s
50 masse_drone = 30 # kg
51 rendement = 0.65 # En général entre 0.6 et 0.7
52 rho = 1.204      # kg/m^3 , à 20 degrés et Patm
53 R = 0.6858       # m , rayon hélice
```

```
55 def puissance(Masse):
56     return 4*(1/rendement)*((Masse*g/4)**3/(2*pi*rho*R**2))**(1/2)
```

```
59 def energie_chemin(Gc,chemin,colis): # Gc graphe complet, chemin=[sommet départ, S1,S2,...,Sn-1], colis=[0,m1,...,mn-1]
60     N=len(Gc)
61     masse=masse_drone+sum(colis)
62     E=0
63     i=0
64     while i < N-1:
65         E += puissance(masse)*Gc[chemin[i]][chemin[i+1]]/Vitesse_moy # Temps du trajet * puissance
66         masse -= colis[i+1]
67         i += 1
68     E += puissance(masse)*Gc[chemin[N-1]][0]/Vitesse_moy # Retour au point de départ
69     return E/Etot # Pourcentage
```

Recherche chemin optimal

```
79 def chemin_opti(Gc,colis):
80     Min=energie_chemin(Gc,[k for k in range(len(Gc))],colis) # chemin zéro [0,1,2,3,...,n-1]
81     ChemOpti=[k for k in range(len(Gc))]
82     Perm=[]
83     permutations(Perm,[k for k in range(1,len(Gc))])
84     for chemin in Perm:
85         if energie_chemin(Gc,chemin,colis) < Min:
86             Min = energie_chemin(Gc,chemin,colis)
87             ChemOpti= chemin
88     return [ChemOpti,Min]
```

```
71 # Modifie la liste perm pour y ajouter les permutations de deb
72 def permutations(perm,deb,fin=[]): # deb = liste des sommets sans le point de départ
73     if len(deb)== 0:
74         perm.append([0] + fin)
75     else:
76         for i in range(len(deb)):
77             permutations(perm,deb[:i] + deb[i+1:], fin + deb[i:i+1])
```

Heuristique de Christofides

```
1  ## Détermination d'un arbre couvrant de poids minimal
2
3  # donne le prochain sommet à relier
4  def plus_proche_sommet(M, distances, tstTraite):
5      min = float("Inf")
6      for v in range(len(M)):
7          if distances[v] < min and not tstTraite[v]:
8              min = distances[v]
9              min_index = v
10     return min_index
11
12
13 # donne la liste des arcs de l'arbre couvrant et leur poids
14 def prim(M): # Matrice d'adjacence complète
15     distances = [float("Inf")] * len(M)
16     distances[0] = 0
17     parent = [None] * len(M)
18     tstTraite = [False] * len(M)
19     parent[0] = -1
20     for _ in range(len(M)):
21         u = plus_proche_sommet(M, distances, tstTraite)
22         tstTraite[u] = True
23         for v in range(len(M)):
24             if distances[v] > M[u][v] > 0 and not tstTraite[v]:
25                 distances[v] = M[u][v]
26                 parent[v] = u
27     return [[parent[i], i, M[i][parent[i]]] for i in range(1, len(M))]
28
29 #liste=[[départ, arrivée, distance]...]
30
31 # donne la matrice d'adjacence de l'arbre couvrant
32 def Arbrecouvrant(M): # Matrice d'adjacence complète
33     M2=[[0 for i in range(len(M))] for i in range(len(M))]
34     for i in prim(M):
35         M2[i[0]][i[1]] = i[2]
36         M2[i[1]][i[0]] = i[2]
37     return M2
```


Heuristique de Christofides

```
39 ## Trouver les sommets de degré impair dans l'arbre couvrant de poids minimal
40
41 def degre_noeud(M,i):    #(arbre couvrant de poids minimal, indice d'un
sommet)
42     v=0
43     for j in M[i]:
44         if j!=0:
45             v+=1
46     if v%2==1:
47         return True      # renvoie True si le degré du sommet est impair
48     else:
49         return False    # Renvoie False sinon
50
```

Heuristique de Christofides

```
52 ## Relier deux à deux les sommets de degré impair (couplage parfait)
53
54 def barycentre(L,masse): #Liste de coordonnées avec masse du colis à déposer
55     abs=[C[L[i]][0]*masse[i] for i in range(len(L))]
56     ord=[C[L[i]][1]*masse[i] for i in range(len(L))]
57     return [sum(abs)/sum(masse),sum(ord)/sum(masse)]
58     # Renvoie le barycentre pondéré par les masses du colis de chaque point
59
60
61 def complete_couvrant(G,L,masse):  #(matrice d'adjacence complète, liste de
62     correspondance des indices entre la matrice de la ville et la matrice
63     complète, masse du colis associé à chaque sommet)
64     M=ArbreCouvrant(G)
65     N=[]
66     for i in range(len(M)):
67         if degre_noeud(M,i):
68             N.append(i)
69     n=len(N)/2
70     for i in range(int(n)):
71         MIN=[distance(C[L[j]],barycentre(L,masse)) for j in N]
72         a=MIN.index(min(MIN))
73         Z=[G[a][j] for j in N[:a]]+[m.inf]+[G[a][j] for j in N[a+1:]]
74         b=Z.index(min(Z))
75         A,B=N[a],N[b]
76         M[A][B],M[B][A]=G[A][B],G[A][B]
77         N.remove(A)
78         N.remove(B)
79     return M # Renvoie un matrice qui contient un cycle eulérien
```

Heuristique de Christofides

```
80  ## Déterminer un trajet optimal
81
82  #chemin=[sommet de départ] on le définit en variable globale
83
84  # Donne les voisins d'un sommet
85  def Voisins(M,i):
86      V=[]
87      for j in range(len(M)):
88          if M[i][j]!=0:
89              V.append(j)
90      return V
91
92
93  # Supprime un arc du graphe
94  def suppressionArc(graphe, sommet1, sommet2):
95      graphe[sommet1].remove(sommet2)
96      graphe[sommet2].remove(sommet1)
97
98
99  # Ajoute un arc au graphe
100  def ajoutArc(graphe, sommet1, sommet2):
101      graphe[sommet1].append(sommet2)
102      graphe[sommet2].append(sommet1)
103
104
105  # Parcours le graphe en profondeur à partir d'un sommet donné
106  def ParcoursProfondeur(graphe,origine,visite):
107      visite[origine] = True
108      for voisin in graphe[origine]:
109          if not visite[voisin]:
110              ParcoursProfondeur(graphe,voisin,visite)
111
```

Heuristique de Christofides

```
112
113 # Permet de savoir si l'arc reliant origine et sommet convient
114 def ArcValide(graphe, origine, voisin):
115     if len(graphe[origine]) == 1:
116         return True
117     else:
118         visiteAvecArc =[False]*(len(graphe))
119         ParcoursProfondeur(graphe, origine, visiteAvecArc)
120         suppressionArc(graphe, origine, voisin)
121         visiteSansArc =[False]*(len(graphe))
122         ParcoursProfondeur(graphe,origine, visiteSansArc)
123         ajoutArc(graphe, origine, voisin)
124         return sum(visiteAvecArc) == sum(visiteSansArc)
125
126
127 # Détermine un cycle eulérien
128 def Cycle_Eulerien(graphe,origine):
129     for voisin in graphe[origine]:
130         if ArcValide(graphe,origine,voisin):
131             chemin.append(voisin)
132             suppressionArc(graphe,origine, voisin)
133             Cycle_Eulerien(graphe,voisin)
134
135
136 # Retire les répétitions d'un chemin donné
137 def coupe_chemin(C):
138     L=[]
139     for i in C:
140         if i not in L:
141             L.append(i)
142     return L
143
```

Références iconographiques

Image drone 1 : <https://fr.pngtree.com/>

Image drone 2 : <https://fr.vecteezy.com/>

Image drone 3 : <https://www.shutterstock.com/>

Image drone 4 : <https://fr.pngtree.com/>

Casier : <https://www.parcelpending.com/>

Image satellite Toulouse : <https://cnes.fr/en/1-loeil-du-satellite/toulouse-ville-rose-et-berceau-de-limage-satellite>

Graphe trainée : Gautier HATTENBERGER

Veine fluide : <https://www.isilf.be/>

Illustrations algorithme de Christofides : <https://fr.wikipedia.org/>